

# Session 09 — Project Guide: useEffect + API Data

This guide explains the actual official Session End for Session 09 in depth. It does not include homework solutions — homework asks you to extend this exact code yourself.

If you forgot most of class, read this guide slowly, section by section, and you should be able to rebuild the mental model from nothing.

---

## 1. Where today's session starts

---

Session 09 starts from Session 08's full homework solution — collection state (`useState<Task[]>`), immutable Add/Toggle/Delete/Edit, callback props, and a validated `AddTaskForm`, on top of everything from Sessions 05–07. If any of that sounds unfamiliar, re-read the Session 08 project guide first — this guide assumes it.

## 2. What was fake about `tasks` and `users`, until today

---

Since Session 07, `tasks` and `users` have always been made-up data, typed directly into `App.tsx`:

```
const initialTasks: Task[] = [
  { id: 1, userId: 1, title: "Finish JavaScript exercise", completed: false },
  { id: 2, userId: 2, title: "Review pull request", completed: true },
  { id: 3, userId: 3, title: "Write session notes", completed: false },
  { id: 4, userId: 1, title: "Update project README", completed: true },
  { id: 5, userId: 2, title: "Fix search bug", completed: false },
  { id: 6, userId: 3, title: "Plan sprint review", completed: true },
];

const users: User[] = [
  { id: 1, name: "Leanne Graham" },
  { id: 2, name: "Ervin Howell" },
  { id: 3, name: "Clementine Bauch" },
];
```

This data was always available INSTANTLY, because it was just sitting in the file. Session 09 replaces both with real data from a real server — and real servers introduce two problems a

hardcoded array never had: **the data might take a moment to arrive**, and **the request might fail**. Everything in this session exists to represent those two new possibilities honestly, as state.

### 3. Why you can't just call `fetch()` in the component body

---

You already know how to fetch data — Phase 1 (Session 3) already used `fetch()`, `async / await`, and `.json()` to load real data in plain JavaScript. The new question this session answers is: WHERE does that code go inside a React component?

Try to imagine calling `fetch()` directly, right inside `App`'s function body, storing the result with `setTasks(...)`. Two problems immediately appear:

1. A component's function body runs DURING RENDER, and its job is to return JSX **synchronously, right now**. `fetch()` doesn't finish "right now" — it finishes later, whenever the network responds.
2. Even if you awaited it somehow, calling `setTasks(...)` triggers a NEW render. If the fetch call were part of the component body, that new render would call `fetch()` again — which finishes and calls `setTasks(...)` again — forever. An infinite loop of fetching.

React's answer is `useEffect`: a way to say "run this code, but not as part of producing JSX — run it as a SIDE EFFECT, separately, after the render happens."

### 4. `useEffect` — the session's central new idea

---

The actual Session End source, in `src/App.tsx`:

```
useEffect(() => {  
  loadTasksData();  
}, []);
```

Two parts:

- **The first argument** is a function containing the side effect itself — "when this effect runs, call `loadTasksData()`."
- **The second argument, `[]`** is the DEPENDENCY ARRAY. At a beginner level: an empty array means "this effect doesn't depend on anything that could change between renders, so only run it once — right after the very first render — and never automatically again."

**Connecting to what you already know:** this is the same "run something once" idea you might expect from a page's `window.onload` in plain JavaScript, just expressed the React way, tied to one specific component instead of the whole page.

## 5. The actual fetch — mostly things you already know

```
async function loadTasksData() {
  setIsLoadingTasks(true);
  setHasTaskError(false);

  try {
    const data = await fetchTasks();
    setTasks(data);
    setIsLoadingTasks(false);
  } catch {
    setHasTaskError(true);
    setIsLoadingTasks(false);
  }
}
```

- `async function` / `await` / `try` / `catch` — all already known from Phase 1. Nothing new syntactically.
- `setIsLoadingTasks(true)` / `setHasTaskError(false)` at the top — before the request even starts, make sure the UI is in a clean "loading" state (this matters for Retry: clicking Retry should clear any old error message immediately, not wait for the new attempt to finish).
- On success: store the real data, turn loading off.
- On failure (the `catch` block runs if `fetchTasks()` throws — for example, no internet connection): record that an error happened, turn loading off.

**An important precision about WHEN `catch` actually runs:** `fetch()` only rejects (which is what triggers `catch`) on a genuine NETWORK failure — the browser couldn't reach a server at all. It does **not** reject just because the server responds with an error status like 404 or 500 — a "page not found" response is still a completed, "successful" fetch as far as the Promise is concerned; detecting that would need an explicit check on the response itself, which is out of scope this session. So if you ever test this by pointing `TASKS_URL` at a slightly wrong but still-real path, don't be surprised if `catch` never runs — that's expected, not a bug. To genuinely see the error path, the URL needs to be unreachable altogether (for example, a hostname that doesn't exist).

`fetchTasks()` itself is a plain function, declared right in `src/App.tsx`, above `App` itself — exactly where `getOwnerName` already lived in Session 07/08:

```
const TASKS_URL = "https://jsonplaceholder.typicode.com/todos?_limit=50";

async function fetchTasks(): Promise<Task[]> {
```

```

const response = await fetch(TASKS_URL);
return (await response.json()) as Task[];
}

```

**Why this stays in `App.tsx` instead of its own file:** splitting fetch logic (and, later, whole components) into separate files is real, valuable organizational work — it's just not this session's job. That reorganization is Session 10's — this session is about the fetch/loading/error PATTERN itself, kept as simple, beginner-readable functions living right alongside everything else in `App.tsx`, the same way the whole file has worked since Session 05.

## 6. Three states, one rendering flow

```

{isLoadingTasks && <p className="message">Loading tasks...</p>}

{!isLoadingTasks && hasTaskError && (
  <div className="message error">
    <p>We could not load the tasks. Please check your internet connection and
try again.</p>
    <button className="retry-button" onClick={loadTasksData}>
      Retry
    </button>
  </div>
)}

{!isLoadingTasks &&
!hasTaskError &&
(visibleTasks.length === 0 ? (
  <p className="empty-state">No tasks to show.</p>
) : (
  <ul className="task-list">
    {visibleTasks.map((task) => {
      const statusText = task.completed ? "Completed" : "Pending";
      const statusClass = task.completed ? "completed" : "pending";

      return (
        <TaskItem
          key={task.id}
          id={task.id}
          title={task.title}
          ownerName={getOwnerName(task.userId)}

```

```

        statusText={statusText}
        statusClass={statusClass}
        onToggle={handleToggle}
        onDelete={handleDelete}
        onSaveEdit={handleSaveEdit}
      />
    );
  })}
</ul>
))}

```

Read this as three questions, asked in order, only one of which can ever be true at once:

1. **Is it still loading?** Show the loading message.
2. **Did it fail?** (only asked once we know it's NOT still loading) Show the error message, with its button calling `loadTasksData` directly — Retry is just "run the exact same load function again," the same function the initial `useEffect` already called.
3. **Otherwise** (loading finished, no error) — ask the exact same question Session 07/08 already asked: is `visibleTasks` empty, or should the real list render? Nothing inside this branch changed at all this session.

**Notice this is all plain, inline JSX — not separate components.** It would be reasonable to pull the loading and error markup out into their own `LoadingMessage` / `ErrorMessage` components (you may even have thought of doing exactly that). This session deliberately keeps them inline: extracting UI into its own component files is real, valuable work that belongs to Session 10's architecture pass, applied consistently to the whole app at once rather than piecemeal. `onClick={loadTasksData}` directly on the button is the callback in action — no separate function needed, since the button already has direct access to `loadTasksData` in the same scope.

## 7. Fetching people too — and why it needs its OWN try/catch

`App.tsx` also gains a `fetchUsers()` function, right next to `fetchTasks`, in exactly the same shape:

```

const USERS_URL = "https://jsonplaceholder.typicode.com/users";

async function fetchUsers(): Promise<User[]> {
  const response = await fetch(USERS_URL);
  return (await response.json()) as User[];
}

```

And inside `App` :

```
const [users, setUsers] = useState<User[]>([]);
const [hasUsersError, setHasUsersError] = useState(false);

async function loadUsersData() {
  setHasUsersError(false);

  try {
    const data = await fetchUsers();
    setUsers(data);
  } catch {
    setHasUsersError(true);
  }
}

useEffect(() => {
  loadTasksData();
  loadUsersData();
}, []);
```

**Why TWO separate functions, each with its OWN `try / catch` , instead of one function**

**fetching both:** if one function tried to fetch tasks AND people and something failed, you couldn't tell which one failed, and a single people-list hiccup would incorrectly break the whole task list too. Separate functions with separate `try / catch` blocks give each resource its own independent loading/error state — the tasks list can succeed while the people list fails, or the other way around, and each half of the screen reacts only to its own outcome.

**Why there's no `isLoadingUsers` :** unlike tasks, the people list doesn't get its own dedicated loading boolean. Instead, "still loading" for people is answered by combining two things already in scope: `users.length === 0` (no data yet) AND `!hasUsersError` (it hasn't failed either):

```
{users.length === 0 && !hasUsersError && (
  <p className="message">Loading people...</p>
)}
{hasUsersError && (
  <div className="message error">
    <p>We could not load the people list. Please check your internet connection
and try again.</p>
    <button className="retry-button" onClick={loadUsersData}>
      Retry
```

```
    </button>
  </div>
  )}
```

This is a smaller, simpler treatment than tasks get, on purpose — the people list is secondary information, not the main content of the page.

`getOwnerName` moves from being a plain module-level function into living INSIDE `App` :

```
function getOwnerName(userId: number): string {
  const user = users.find(function (user) {
    return user.id === userId;
  });

  if (user) {
    return user.name;
  }

  return "Unknown person";
}
```

**Why it has to move:** it used to read a module-level `const users` array that existed outside any component. Now `users` is STATE, which only exists inside `App` — so any function reading it has to live where that state is in scope, exactly like every other handler in this file.

## 8. The `AddTaskForm` problem nobody saw coming

Session 08's `AddTaskForm` picked a default owner like this:

```
const defaultUserId = props.selectedUserId || props.users[0].id;
const [draftUserId, setDraftUserId] = useState(defaultUserId);
```

The moment `users` becomes fetched state starting at `[]`, this breaks — badly. On `AddTaskForm`'s very first render, `users` is **always** `[]` (React finishes the first render before any effect, and therefore any fetch, has a chance to run). `props.users[0]` is `undefined` on that first render, and reading `.id` off `undefined` throws immediately — crashing the whole page before a single fetch even starts.

The actual Session End fix:

```

const [draftUserId, setDraftUserId] = useState(0);

function getDefaultUserId(): number {
  if (props.selectedUserId !== 0) {
    return props.selectedUserId;
  }

  if (props.users.length > 0) {
    return props.users[0].id;
  }

  return 0;
}

const ownerIdToUse = draftUserId !== 0 ? draftUserId : getDefaultUserId();

```

- `0` is used as a sentinel meaning "no explicit choice made yet" — reusing the exact meaning `0` already has for `selectedUserId` elsewhere in this app ("All people").
- `getDefaultUserId()` checks `props.users.length > 0` BEFORE touching `props.users[0]` — no crash possible.
- `ownerIdToUse` is a **derived value**, recalculated on every render from CURRENT props — the same "calculate it fresh, don't store a stale copy" idea from Session 07's derived data, now solving a real bug instead of just avoiding one.

**Why deriving fresh every render fixes it, when a one-time `useState` initializer couldn't:** a `useState(x)` initializer only ever runs ONCE, at the component's first mount — it captures whatever `x` was at that moment forever, even if the real value that `x` depends on changes later. `ownerIdToUse`, by contrast, is recalculated from scratch on every single render. The moment `App` re-renders with a real, populated `users` array (after the fetch resolves), `AddTaskForm` re-renders too, and `ownerIdToUse` automatically resolves to a real person's id — with no `useEffect` needed to "notice" the change.

`handleSubmit` also gained one more early guard:

```

if (props.users.length === 0) {
  return;
}

```

If there's genuinely no one to assign a task to yet, submitting does nothing — instead of creating a task with a meaningless owner id.



## 9. What did NOT need to change

---

Every CRUD handler from Session 08 — `handleAddTask`, `handleToggle`, `handleDelete`, `handleSaveEdit` — is **completely unchanged**. Every status/search/person filter, every derived stat, `peopleWithCounts`, the empty-result message — all unchanged. None of that code ever cared WHERE `tasks` / `users` came from, only that they're arrays in state. That's the entire payoff of building on top of state and derived data since Sessions 06–08: swapping the DATA SOURCE underneath required touching almost nothing else.

---

## Glossary

---

- **Side effect** — something a component does that isn't part of producing its JSX output — fetching data, for example. Side effects don't belong in a component's function body; they belong in `useEffect`.
- **`useEffect`** — tells React to run a function after rendering, separately from rendering itself.
- **Dependency array ( `[]` )** — `useEffect`'s second argument. An empty array means "run once, right after the first render, never again automatically."
- **Independent resource states** — separate loading/error tracking per piece of fetched data, so one resource's failure never affects another's success.
- **Retry** — re-running the exact same load function that failed, triggered by a button instead of automatically.

## Common mistakes

---

```
// WRONG – fetches during render, causes an infinite loop
function App() {
  fetchTasks().then(setTasks);
  ...
}

// RIGHT – the fetch is a side effect, run once via useEffect
useEffect(() => {
  loadTasksData();
}, []);
```

```
// WRONG – crashes the instant users is still empty
const defaultUserId = props.selectedUserId || props.users[0].id;
```

```
// RIGHT – checks length before touching an array index
function getDefaultUserId(): number {
  if (props.selectedUserId !== 0) {
    return props.selectedUserId;
  }
  if (props.users.length > 0) {
    return props.users[0].id;
  }
  return 0;
}
```

## Review questions

---

1. Why can't a component just call `fetch()` directly inside its own function body?
2. What does the empty dependency array `[]` mean for `useEffect`, at a beginner level?
3. What does `try / catch` accomplish inside `loadTasksData`, in your own words?
4. `fetch()` doesn't reject for an HTTP 404 or 500 response. So what actually causes `catch` to run?
5. Why do `tasks` and `users` each need their OWN `try / catch`, instead of sharing one?
6. Why is there no `isLoadingUsers`, even though there IS an `isLoadingTasks`?
7. Why did `AddTaskForm`'s old `defaultUserId` line crash once `users` became fetched state?
8. Why does recalculating `ownerIdToUse` on every render fix that crash, when a `useState` initializer couldn't?
9. Why did `getOwnerName` have to move inside `App`?
10. Why didn't any of Session 08's CRUD handlers need to change this session?

## Answers

---

1. A component's function body runs during render and must return JSX synchronously right now; `fetch()` finishes later. Calling it directly (and updating state with the result) inside the body would also re-trigger on every render, causing an infinite fetch loop.
2. Run this effect once, right after the component's first render, and never automatically again — there's nothing in the array that could change to justify running it again.
3. It attempts the fetch; if it succeeds, real data is stored and loading turns off; if it throws (for example, no network), the error state is set and loading turns off — either way, the app ends up in a clean, known state.
4. A genuine network failure — the browser couldn't reach a server at all (no connection, a nonexistent hostname, etc.). An HTTP error status like 404 or 500 is still a completed fetch as

far as the Promise is concerned and does not trigger `catch` on its own.

5. So each resource's loading/error state is completely independent — one can fail while the other succeeds, and neither's failure incorrectly affects the other's rendering.
6. The people list is secondary information; "still loading" for people is answered by combining `users.length === 0` with `!hasUsersError`, without needing a dedicated boolean.
7. Because on `AddTaskForm`'s first render, `users` is always `[]` (rendering always happens before any effect/fetch runs) — `props.users[0]` was `undefined`, and reading `.id` off `undefined` throws immediately.
8. A `useState` initializer only runs once, at first mount, and never re-evaluates. A value recalculated fresh on every render automatically reflects the CURRENT props — once `users` populates, the very next render already sees the correct default, with no effect needed.
9. It used to read a module-level `const users` array that no longer exists; now `users` is state that only exists inside `App`, so any function reading it must live where that state is in scope.
10. Because they only ever depended on `tasks` being "an array in state" — never on where that array came from. Swapping a hardcoded seed for fetched data didn't change that fact at all.

## What you should be able to explain now

---

- ☐ Why fetching can't happen directly inside a component's function body.
- ☐ What `useEffect` with an empty dependency array does and when it runs.
- ☐ The full loading → error → content rendering flow, and why the three checks must be asked in that order.
- ☐ Why `tasks` and `users` each need independent loading/error tracking.
- ☐ Why `AddTaskForm`'s old default-owner logic broke, and how deriving the value fresh every render fixed it.
- ☐ Why Session 08's CRUD code needed zero changes this session.

## Not yet

---

These ideas are real, and some are close relatives of what you just learned — but none are taught yet. If you find yourself trying to use one, stop — it belongs to a later session.

- **A second `useEffect` with a real dependency (not `[]`)** — not used anywhere in this course yet.
- **Hiding or removing `AddTaskForm` while people are loading (mount-gating)** — a later session; this session's fix keeps the form always rendered and only guards its internal logic.
- **Editing a task's owner, delete confirmation, keyboard shortcuts for editing, resetting all filters with one button** — later sessions.

- **Splitting `App` into many smaller files, or destructuring props** — a later session. Every component in this session still reads props as `props.title` , `props.onDelete` , etc., on purpose.
- **React Fragment, or extracting shared types into their own file** — later sessions.